COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	10/16/26	**DATE SUBMITTED:**	10/19 /26
NAME:	Reyes, Cyrill Joferson B. Silvoza, Keith Nicole B.	**INSTRUCTOR:**	Engr. Jimlord M.Quejado

ACTIVITY NO. 13.1 **Parallel Algorithm and Multithreading**

1. PROCEDURE OUTPUT

Reyes

Screenshot

CPE010_HOA13p1_Reyes_sc1.cpp

```cpp
#include <iostream>
#include <string>
#include <thread>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();
    return 0;
}
```

Run CPE010_HOA13p1_Reyes_sc1.cpp

```
"C:\Users\Jofjof\OneDrive\Documents\CPE12S4 - Reyes\HOA13p1\CPE010_HOA13p1_Reyes_sc1.exe"
Printing integer: 10
Printing string: T.I.P.

Process finished with exit code 0
```

Analysis

This part shows how to create threads using C++11. The print function is given to the thread object t1 along with the number 10 and the text "T.I.P." as arguments. The execution of the main thread is stopped when t1.join() is called. If you don't use .join(), the main thread will end before t1 finishes, which causes a runtime error.

Table 13-1. Simple One-Threaded Example

Screenshot

CPE010_HOA13p1_Reyes_sc1.cpp | CPE010_HOA13p1_Reyes_sc2.cpp

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <thread>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + ":" + str;
    std::cout << msg << std::endl;
}

int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };

    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back( x: std::thread( [>>] print, [>>] i, [>>] s[i]));
    }

    for (auto &th :thread & : threads) {
        th.join();
    }

    return 0;
```

Threads & Variables | Console | Memory View | GDB | Live Watches

```
"C:\Users\Jofjof\OneDrive\Documents\CPE12S4 - Reyes\HOA13p1\CPE010_HOA13p1_Reyes_sc4.exe"
Original Array: 38 27 43 3 9 82 10 19 50 12 7 64

Sorted Array:   3 7 9 10 12 19 27 38 43 50 64 82

Process finished with exit code 0
```

Analysis

This program demonstrates concurrency by creating worker threads inside a for loop. Each thread executes print() concurrently on separate vector elements. A second loop calls .join() on every thread object to synchronize execution before the main function terminates. Constructing string msg before calling std::cout prevents character output overlapping during parallel operations.

Table 13-2. Multithreaded Example.

Silvoza

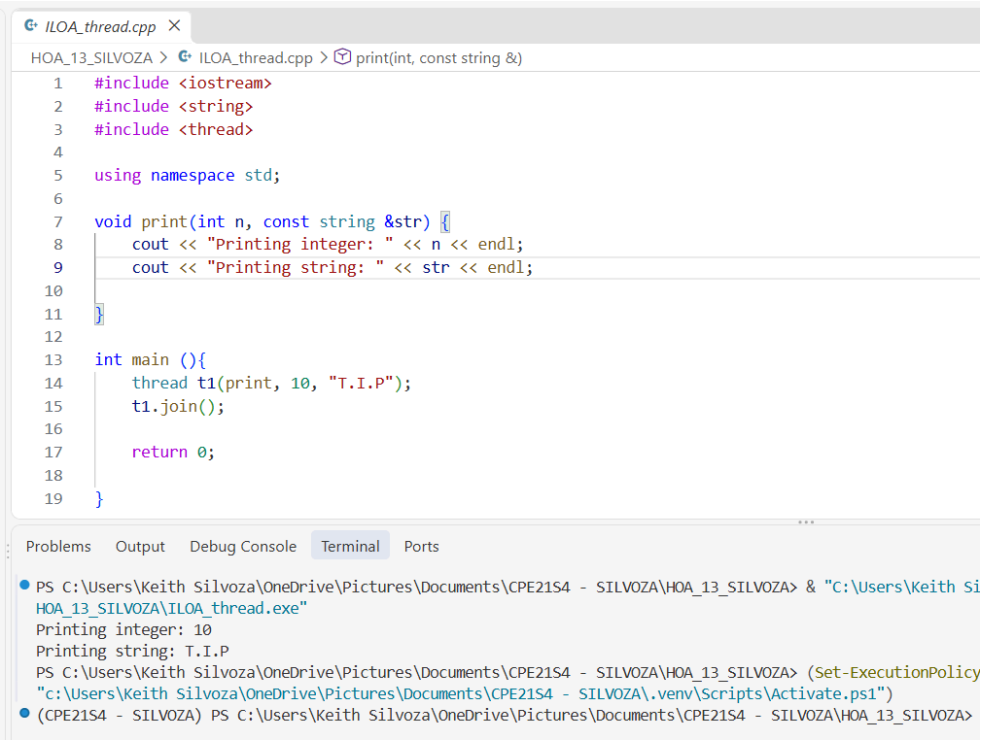
Screenshot	
Analysis	THE SINGLE-THREADED CODE PRODUCES CLEAN, PREDICTABLE OUTPUT EVERY TIME, PRINTING THE INTEGER AND STRING IN EXACT ORDER. CALLING t1.join() PAUSES THE MAIN THREAD UNTIL t1 FINISHES, KEEPING THE PROGRAM FROM EXITING EARLY AND CRASHING. SINCE ONLY ONE WORKER THREAD IS RUNNING, EXECUTION STAYS STRICTLY LINEAR WITHOUT ANY THREAD COMPETITION.

Table 13-1. Simple One-Threaded Example

Screenshot

ILOB_multithreading.cpp

HOA_13_SILVOZA > ILOB_multithreading.cpp > ...

```cpp

#include <iostream>
#include <string>
#include <vector>
#include <thread>
using namespace std;

void print(int n, const string &str) {
    string msg = to_string(n) + " : " + str;
    cout << msg << endl;
}

int main() {
    vector<string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };

    vector<thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(thread(print, i, s[i]));
    }

    for (auto &th : threads) {
        th.join();
    }
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA\HOA_13_SILV
3 : Engineers2 : Computer0 : T.I.P.


1 : Competent
PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA\HOA_13_SILV
ocuments\CPE21S4 - SILVOZA\.venv\Scripts\Activate.ps1")
(CPE21S4 - SILVOZA) PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 -
tithreading.exe"
0 : T.I.P.
2 : Computer
3 : Engineers
1 : Competent
(CPE21S4 - SILVOZA) PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 -
tithreading.exe"
0 : T.I.P.3 : Engineers
1 : Competent
2 : Computer
```

Analysis

RUNNING MULTIPLE THREADS CONCURRENTLY CAUSES SCRAMBLED OUTPUT—LIKE 3 : Engineers2 : Computer0 : T.I.P.BECAUSE ALL FOUR THREADS TRY WRITING TO std::cout AT THE EXACT SAME TIME. THE PRINT ORDER ALSO CHANGES BETWEEN RUNS SINCE THE OS SCHEDULER DECIDES WHEN EACH THREAD GETS CPU TIME. FINALLY, THE th.join() LOOP MAKES SURE EVERY BACKGROUND THREAD FINISHES ITS TASK CLEANLY BEFORE main() CLOSES.

Table 13-2. Multithreaded Example.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Reyes

PART A:

Questions:

1. Write a definition of multithreading and its advantages/disadvantages.

Definition:

- Multithreading is a method in C++ programming where multiple thread objects (std::thread) are created and controlled inside a single program to run different smaller tasks at the same time.

Advantages:

- **Resource Efficiency:** Threads within the same process share the same memory space, resulting in lower overhead than spawning separate processes.
- **Improved CPU Core Utilization:** Distributes execution workloads across available multi-core processors.
- **Enhanced Responsiveness:** Keeps primary threads (such as user interfaces) responsive while offloading intensive processing to worker threads.

Disadvantages:

- **Data Races:** Occurs when two or more threads concurrently access a shared memory location and at least one thread modifies it without synchronization, leading to undefined behavior.
- **Creation & Management Overhead:** Creating, context switching, and destroying threads costs CPU time and memory overhead, making it inefficient for trivial functions.
- **Debugging Complexity:** Non-deterministic execution orders introduce complex issues like race conditions and deadlocks that are difficult to reproduce.

2. Use-Cases for Multithreading

- **Web and Application Servers:** Spawns separate worker threads to handle thousands of concurrent client connections and database requests simultaneously without blocking incoming traffic.
- **Game Development:** Separates decoupled tasks—such as rendering graphics on one thread, processing physics/AI on another thread, and handling audio streams on a third thread—to maintain target frame rates.
- **Desktop Application GUI:** Offloads heavy background operations (such as video rendering, file compression, or downloading) to a background thread to prevent the user interface from freezing.

3. Differentiate between parallelism and concurrency.

Concurrency

- is when two or more tasks overlap in execution and communicate with each other, typically sharing memory locations or resources on a single machine. It focuses on task structure and can occur on a single core via task switching or across multiple cores.

Parallelism

- is when multiple copies of a program or task run simultaneously at the exact same physical moment, executing on separate CPU cores with independent datasets.

PART B:

CPE010_HOA13p1_Reyes_sc1.cpp CPE010_HOA13p1_Reyes_sc2.cpp CPE010_HOA13p1_Reyes_sc3.cpp

```cpp
#include <iostream>
#include <thread>
#include <chrono>

// Global variable of type integer
int global_var = 0;

// Function that adds a given value to the global variable
void add(int val) {
    // Small delay to simulate processing time across threads
    std::this_thread::sleep_for( rtime: std::chrono::milliseconds( rep: 50));
    global_var += val;
}

int main() {
    // Create 3 threads and pass the add function with different integer values
    std::thread T1( [>>] add, 10);
    std::thread T2( [>>] add, 20);
    std::thread T3( [>>] add, 30);

    std::cout << "Display 1 (Before T1.join()): Global Variable = " << global_var << std::endl;
    T1.join();

    std::cout << "Display 2 (After T1.join()):  Global Variable = " << global_var << std::endl;
    T2.join();

    std::cout << "Display 3 (After T2.join()):  Global Variable = " << global_var << std::endl;
    T3.join();

    std::cout << "Display 4 (After T3.join()):  Global Variable = " << global_var << std::endl;

    return 0;
}
```

Analysis:

- The program starts three threads named T1, T2, and T3.Each thread adds a different value—10, 20, and 30 respectively—to a shared global variable that begins at 0. Showing the global variable before and after every .join() call reveals how its value goes up step by step from 0 to 10, then to 30, and finally reaches 60. By stopping the main thread,.join() ensures that execution is synchronized until all worker threads have completed their assigned addition.

PART C:

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <functional>

// Standard Merge function to combine two sorted sub-arrays
void merge(std::vector<int>& arr, int left, int mid, int right) {
    std::vector<int> temp(n:right - left + 1);
    int i = left, j = mid + 1, k = 0;

    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
        }
    }
    while (i <= mid) temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];

    for (int p = 0; p < k; p++) {
        arr[left + p] = temp[p];
    }
}

// Parallel Merge Sort implementation
void parallelMergeSort(std::vector<int>& arr, int left, int right, int depth = 0) {
    if (left >= right) return;
```

```
    int mid = left + (right - left) / 2;

    // Limit thread creation depth (thresholding) to minimize thread management overhead
    if (depth < 2) {
        // Spawn separate threads for the left and right sub-arrays
        std::thread t1( f: [>>] parallelMergeSort, std::ref( [&] arr), [>>] left, [>>] mid, depth + 1);
        std::thread t2( f: [>>] parallelMergeSort, std::ref( [&] arr), mid + 1, [>>] right, depth + 1);

        // Wait for both sub-threads to complete sorting their respective sub-arrays
        t1.join();
        t2.join();
    } else {
        // Sequential fallback for deeper subproblems
        parallelMergeSort( [&] arr, left, right: mid, depth + 1);
        parallelMergeSort( [&] arr, left: mid + 1, right, depth + 1);
    }

    // Merge the sorted halves
    merge( [&] arr, left, mid, right);
}
```

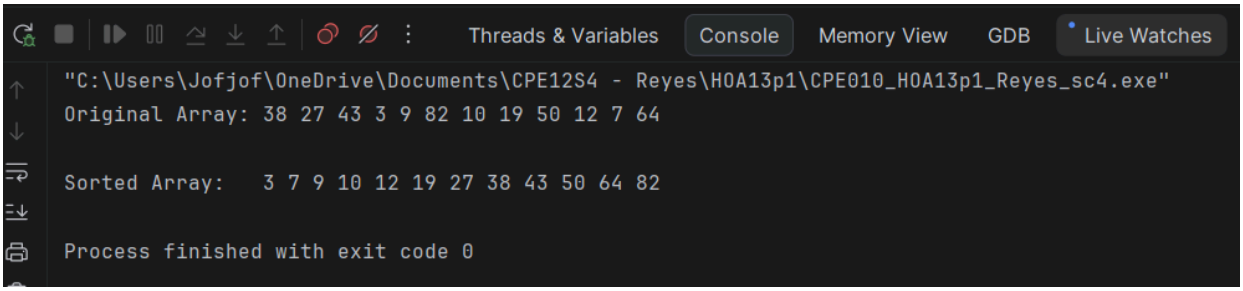
```
int main() {
    std::vector<int> data = {38, 27, 43, 3, 9, 82, 10, 19, 50, 12, 7, 64};

    std::cout << "Original Array: ";
    for (int num : data) {
        std::cout << num << " ";
    }
    std::cout << "\n\n";

    // Execute Parallel Merge Sort
    parallelMergeSort( [&] data, left: 0, right: data.size() - 1);

    std::cout << "Sorted Array:   ";
    for (int num : data) {
        std::cout << num << " ";
    }
    std::cout << "\n";

    return 0;
}
```



Analysis:

- The Merge Sort algorithm was parallelized by spawning separate threads to sort the left and right sub-array partitions concurrently, which avoids data races because each thread operates on independent memory ranges. Calling `.join()` synchronizes execution so both sub-arrays complete sorting before they are merged together, while a depth limit prevents excessive thread creation overhead. The Merge Sort algorithm was parallelized by spawning separate threads to sort the left and right sub-array partitions concurrently, which avoids data races because each thread operates on independent memory ranges. Calling `.join()` synchronizes execution so both sub-arrays complete sorting before they are merged together, while a depth limit prevents excessive thread creation overhead.

Summary of Lessons Learned: In this activity, I learned how to implement multithreading and concurrency in C++ using the <thread> library. I got hands-on experience spawning worker threads, passing arguments, and using .join() to keep the main thread synchronized. I also got a much clearer understanding of the difference between concurrency and parallelism, as well as the risks of data races when multiple threads access shared memory.

Analysis of the Procedure: In Part 1, I observed how a single worker thread runs and why .join() is essential to prevent the main thread from terminating early. In Part 2, I created a vector of threads inside a loop to process string data concurrently. This clearly showed how multithreading takes advantage of multi-core CPU resources to handle multiple operations faster than standard sequential execution.

Analysis of the Supplementary Activity: Part A reinforced the theoretical foundation, advantages, and real-world use cases of multithreading. Part B helped me visualize thread synchronization by tracking how a global variable updated step-by-step as each .join() was called. In Part C, I successfully applied multithreading to Merge Sort, proving that divide-and-conquer algorithms can be parallelized cleanly because each thread operates on independent sub-array ranges.

Concluding Statement / Feedback: Overall, I feel I did really well in this activity since I successfully got all the code working and grasped how thread synchronization works. My main area for improvement is getting better at configuring compiler settings and local IDE toolchains (like setting up MinGW POSIX threads) so that C++ concurrency libraries compile without setup issues.

SIlvoza

PART A:

1. Write a definition of multithreading and its advantages/disadvantages.

Definition:

- Multithreading is the ability of a single process to execute multiple threads concurrently, where each thread is a lightweight unit of execution sharing the same memory space. This technique enhances performance, responsiveness, and resource utilization, especially on multi-core processors.

Advantages:

- Faster Speed: Heavy tasks finish much quicker because work is split across multiple threads.
- Better CPU Use: Takes full advantage of multi-core processors so CPU power isn't wasted.
- Keeps Apps Responsive: Heavy tasks run in the background without freezing the app or user interface.

Disadvantages:

- Data Races: Multiple threads changing the same variable at the same time can corrupt data or cause unexpected results.
- Hard to Debug: Output order changes almost every run, making bugs really tricky to find and fix.
- Extra Overhead: Creating and managing threads takes extra memory and setup time, which can actually slow down tiny tasks.

2. Rationalize the use of multithreading by providing at least 3 use-cases.

- Web Browsers: Running different tabs on separate background threads so one frozen page doesn't crash your entire browser.
- Video Games: Handling graphics, player physics, and background audio at the same time on different threads for smooth gameplay.
- App Downloads: Downloading a large file in the background while keeping the app buttons and screen smooth and clickable.

3. Differentiate between parallelism and concurrency.

Parallelism:

- Like multiple cashiers serving multiple customers at the same time (e.g. lines of people taking turns using 1 coffee machine.)

Concurrency :

- Like a single cashier serving multiple customers by switching between them very quickly(2 lines of people using 2 separate coffee machines at once.)

PART B:

COMPUTER ENGINEERING

QUEZON CITY

Code

part_b.cpp | ILOA_thread.cpp | ILOB_multithreading.cpp | part_c.cpp

HOA_13_SILVOZA > part_b.cpp > main()

```cpp
#include <iostream>

int global_var = 0;
void add(int value) {
    global_var += value;
}

int main() {
    std::thread T1(add, 10);
    std::thread T2(add, 20);
    std::thread T3(add, 30);

    // DISPLAY
    std::cout << "Initial Display: " << global_var << std::endl;

    // JOIN T1
    T1.join();
    std::cout << "Display after T1.join(): " << global_var << std::endl;

    // JOIN T2
    T2.join();
    std::cout << "Display after T2.join(): " << global_var << std::endl;

    // JOIN T3
    T3.join();
    std::cout << "Display after T3.join(): " << global_var << std::endl;

    return 0;
}
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA\HOA_13_S
Initial Display: 40
Display after T1.join(): 60
Display after T2.join(): 60
Display after T3.join(): 60
PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA\HOA_13_S
ve\Pictures\Documents\CPE21S4 - SILVOZA\.venv\Scripts\Activate.ps1")
(CPE21S4 - SILVOZA) PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S
```

Analysis

The output shows 60 across all lines because all three background threads completed their addition tasks ($10 + 20 + 30 = 60$) before the main() thread executed its very first print statement. Because thread timing is controlled non-deterministically by the OS scheduler, the background threads executed faster than main() during this specific run. Using .join() guarantees that main() pauses to wait for each thread to finish completely before exiting.

COMPUTER ENGINEERING

QUEZON CITY

PART C:

Code	part_b.cpp · main_ILO_C.cpp · part_c.cpp
	HOA_13_SILVOZA > part_c.cpp > ...
	1 #include <iostream>
	2 #include <thread>
	3
	4 using namespace std;
	5
	6 int sum1 = 0;
	7 int sum2 = 0;
	8 int arr[6] = {10, 20, 30, 40, 50, 60};
	9
	10 void addFirstHalf() {
	11 for (int i = 0; i < 3; i++) {
	12 sum1 += arr[i];
	13 }
	14 }
	15
	16 void addSecondHalf() {
	17 for (int i = 3; i < 6; i++) {
	18 sum2 += arr[i];
	19 }
	20 }
	21
	22 int main() {
	23 thread t1(addFirstHalf);
	24 thread t2(addSecondHalf);
	25
	26 t1.join();
	27 t2.join();
	28
	29 int total = sum1 + sum2;
	30
	31 cout << "First half sum: " << sum1 << endl;
	32 cout << "Second half sum: " << sum2 << endl;
	33 cout << "Total sum: " << total << endl;
	34
	35 return 0;
	36 }
	Problems Output Debug Console Terminal Ports
	PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE2
	pters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIE
	oft-MIEngine-Pid-zq4wmlrl.4gm' '--dbgExe=D:\mingw64\bin\gd
	First half sum: 60
	Second half sum: 150
	Total sum: 210
	PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE2
	uments\CPE21S4 - SILVOZA\.venv\Scripts\Activate.ps1")
	(CPE21S4 - SILVOZA) PS C:\Users\Keith Silvoza\OneDrive\Pic

Analysis	This code divides array summation between two threads, where t1 calculates the first half and t2 calculates the second half simultaneously. Using separate variables (sum1 and sum2) prevents data races during concurrent execution. Calling .join() ensures main() waits for both threads to complete before adding their partial sums together to output the final total.

Lessons Learned:

- Learned how to spawn and manage concurrent execution using C++ std::thread.
- Understood that calling .join() forces main() to pause and wait for background threads to finish.
- Identified that multiple threads modifying a single shared variable without locks causes data races.

Analysis of the Procedure

The procedure demonstrated basic thread synchronization and timing behavior. In Part B, observing how global variable values updated showed that thread execution speed depends on the OS scheduler, emphasizing why synchronization control is necessary.

Analysis of the Supplementary Activity

The activity applied multithreading to parallel array summation by dividing the array across two threads. Having each thread calculate into its own separate variable safely avoided data races while showing how concurrency improves performance.

Concluding Statement / Feedback

I performed well in writing the thread logic, running the programs in VS Code, and analyzing thread outputs. To improve further, I need to practice using explicit synchronization tools like std::mutex to safely manage shared memory in more complex programs.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: *ChatGPT*

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code assistance, explanation of concepts analysis assistance

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: AI was used to clarify multithreading concepts, structure C++ code, and draft preliminary analysis summaries. All final code executions, lab outputs, and final analytical evaluations reflect my own work and understanding.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.